

CuKD-XAI: Compressing WSN Intrusion Detectors through Knowledge Distillation, Explanation Auditing, and Fixed-Point Validation

Nishant Harkut

Abstract—High-capacity intrusion detectors are difficult to place on wireless sensor and edge devices, yet compression studies often stop at parameter counts or desktop accuracy. This paper presents CuKD-XAI, a controlled framework for testing curriculum-guided knowledge distillation with explainable artificial intelligence across five questions: predictive compression, training-route effects, explanation agreement, numerical deployment fidelity, and cross-dataset robustness. Under the archived five-class WSN-DS protocol, a 1,189-parameter student distilled from a calibrated random forest attains 0.91997 ± 0.00312 macro-F1 over ten seeds using 4.64 KB of FP32 parameters; a 3,397-parameter two-teacher student attains 0.93353 ± 0.01136 in 13.27 KB. These students are respectively $58.8\times$ and $20.6\times$ smaller than the full neural teacher on the same parameter-storage basis. Curriculum-based routes are not uniformly beneficial, and the audited curriculum-KD Student A does not align with the RF reference’s global SHAP ranking ($\rho = 0.0466$). Fixed-point RF-distilled exports reproduce every generated integer reference in four complete 56,200-record replays on ESP32-C3 and Arduino UNO R4 WiFi, totaling 224,800 board predictions; mean on-device preprocessing-plus-inference latency is $118.40\text{--}791.57 \mu\text{s}$. On Edge-IIoTset, the best compact macro-F1 changes from 0.7111 under a strict 43-feature protocol to 0.8244 under a 49-feature protocol, exposing sensitivity to task construction. The evidence establishes KB-scale executable model cores while showing that predictive gain, cross-model explanation agreement, and conversion fidelity are independent outcomes. Archived multi-seed WSN-DS tables retain pre-split standardization; a train-only-scaler seed-42 confirmation regenerates RF-KD deployment, ONNX/OpenVINO runtime, fixed-point export, and four full-test HIL replays under train-fitted scaling. A five-seed exact-feature-group package documents group-disjoint sensitivity under train-only scaling.

Index Terms—wireless sensor networks, intrusion detection, knowledge distillation, model compression, explainable artificial intelligence, SHAP, TinyML, fixed-point inference, hardware-in-the-loop, Edge-IIoTset

I. INTRODUCTION

WIRELESS sensor network (WSN) intrusion detection couples an imbalanced classification problem with an embedded-systems constraint. A detector must recognize minority routing and traffic attacks, but its memory, arithmetic, and latency costs must also fit the device that will execute it. Recent reviews continue to identify this accuracy–resource tension as a deployment barrier [1]. On WSN-DS, resampling, feature selection, and optimized tree ensembles now report accuracies above 99.7% [2]–[5]. Those results establish the

predictive ceiling of the dataset; faithful kilobyte-scale integer representation remains a separate question.

Knowledge distillation (KD) offers a direct compression mechanism: a compact student learns from the class distribution of a higher-capacity teacher in addition to hard labels [6]. CuKD-XAI expands to *Curriculum-guided Knowledge Distillation with Explainable Artificial Intelligence*. Curriculum benefit is a tested hypothesis within the framework. A calibrated random forest provides a strong tabular teacher, a full multilayer perceptron (MLP) provides a neural teacher, and curriculum variants test whether example ordering improves transfer. The students remain ordinary two-hidden-layer MLPs, making model capacity the primary control on deployment cost.

Deployment evaluation requires four distinct quantities. Predictive performance may survive while minority-class behavior changes; a student may agree with labels while ranking features differently from its teacher; a converted integer kernel may drift from FP32 despite matching its own reference exactly; and a small model core may still sit inside a much larger framework-linked firmware image. Explanation disagreement [7], [8] and integer-conversion error must therefore be measured independently of model size.

CuKD-XAI studies these distinctions through five research questions:

- RQ1: What predictive performance is retained when RF and neural WSN detectors are compressed into 1,189- and 3,397-parameter students?
- RQ2: Do RF-KD, MLP-KD, curriculum-KD, random pacing, imbalance-aware teachers, or two-teacher co-distillation consistently improve the compact students?
- RQ3: How closely does the preserved compact Student A align with the high-performing RF reference’s SHAP feature ranking?
- RQ4: How faithfully and efficiently do selected students execute after software export and fixed-point conversion on MCU-class boards?
- RQ5: How does the compact-student strategy behave on a larger 15-class Edge-IIoTset task under two explicit preprocessing protocols?

The resulting study makes five contributions:

- It evaluates two compact capacities over ten WSN-DS seeds across scratch training, RF and neural KD, curriculum and random-pacing controls, imbalance-aware

Nishant Harkut is with Atal Bihari Vajpayee Indian Institute of Information Technology and Management, Gwalior, India (e-mail: img_2023040@iiitm.ac.in).

teacher training, and two-teacher distillation.

- It reports compression on consistent and explicitly different bases: FP32 parameters, ONNX graphs, fixed-point constants, complete board compilation, and MSP430F1611 cross-compilation.
- It treats cross-model explanation agreement as a measured outcome by comparing RF-reference and compact-student SHAP feature rankings, including a negative result.
- It validates fixed-point Student A and B kernels on ESP32-C3 and Arduino UNO R4 WiFi over the complete 56,200-record test replay, with sequence integrity, latency distributions, FP32 drift, and compile deltas.
- It repeats the compact-model study on Edge-IIoTset under two documented protocols, separating cross-dataset robustness from direct WSN-DS performance.

The central contribution is an evidence chain showing how much minority-sensitive performance small executable students retain, where explanation rankings diverge, and how software conversion affects numerical and hardware behavior.

II. RELATED WORK

A. Accuracy-Oriented WSN-DS Intrusion Detection

WSN-DS models Normal traffic and Blackhole, Flooding, Grayhole, and TDMA scheduling attacks in a LEACH-based simulation [9]. Imbalance correction and ensembles have raised its predictive ceiling. MLSTL-WSN combines SMOTE–Tomek processing, LightGBM, SHAP, and recursive feature elimination and reports 99.92% multiclass accuracy [2]. Talukder *et al.* combine KMeans-SMOTE, PCA, and RF classification and report 99.94% accuracy and F1 [3]. Biraahim *et al.* use PSO, SMOTE–Tomek, RF/decision-tree/KNN ensembling, SHAP, and LIME, reporting 99.73% accuracy [4]. Tabu-optimized RF [5] and GSWO-CatBoost [10] further demonstrate the strength of tree models on this dataset.

Earlier WSN-DS studies also show why overall accuracy is an incomplete comparison. GXGBoost reports 10-fold class detection rates from 92.9% for scheduling to 99.9% for Normal traffic, explicitly optimizing minority-class behavior [11]. A CNN–LSTM study reports 0.944 accuracy, 0.959 precision, and 0.922 recall under its own 25-epoch protocol [12]. These protocols differ from the fixed split used here, but their class-sensitive reporting motivates the per-class and macro-averaged analysis used in our experiments.

These methods answer the detection question but generally leave model-core size, firmware equivalence, and on-board execution outside their experimental scope. Xiao and Duan explicitly frame a DNN/CatBoost ensemble as resource-aware, reporting 95.62% test accuracy and 96.07% F1, but their ensemble remains a different deployment target from a KB-scale dense student [13]. Alfarrar and AbuSamra provide a closer system comparator: an integer sensor-side filter and pruned INT8 CNN–LSTM gateway model, with reported 98% accuracy, 0.93 macro-F1, 42 ms, and 28 mJ for their evaluated window [14]. Their study provides energy and lifetime evidence; ours covers five WSN-DS labels through direct MCU model-core execution, with radio integration and energy measurement left for system-level evaluation.

B. Compression, Quantization, and Embedded IDS

KD was introduced as a way to transfer class relationships from a high-capacity model to a compact model [6]. Curriculum learning changes the order or availability of examples during optimization [15]; self-KD has also been explored for lightweight network IDS [16]. Benaddi *et al.* combine SHAP-guided pruning, KD, and Kronecker layers on TON_IoT, reporting a 3,042-parameter, 22.29 KB student with 0.9863 macro-F1 [17]. The different dataset precludes a direct score comparison, while the joint treatment of compression and IDS interpretation remains closely relevant.

KD-based IDS now spans broader operating assumptions. Wisanwanichthan and Thammawichai report 92–95% parameter reduction and 7–11% faster inference across five IoT/UAV datasets [18]. Hossain *et al.* combine federated learning, SHAP-based feature selection, and KD for IoT botnet detection [19], while FD-IDS introduces proximal regularization and KD for non-IID Edge-IIoTset and N-BaIoT clients [20]. LENS-XAI combines KD, variational autoencoding, and attribution analysis across four datasets [21]. Their federated, feature-selection, and dataset contracts complement the centralized WSN compression experiment considered here.

Quantization-aware and integer-only inference are established mechanisms for moving neural networks to constrained hardware [22], [23]. Misrak and Melaku evaluate dynamic and quantization-aware IDS conversion, while Javed *et al.* embed quantized tree detectors on ESP32 smart thermostats [24], [25]. These studies reinforce target-runtime measurement because operator support and framework overhead can make lower-precision execution slower.

C. Explanation Fidelity

SHAP assigns additive Shapley-value attributions to model outputs [26]. Existing WSN-DS studies use SHAP to rank or select features [2], [4]. CuKD-XAI asks a different question: whether a preserved compact student and the high-performing RF reference rank the unchanged 17-feature input space similarly. This is a cross-model explanation audit, not a feature-selection or explanation-transfer claim. The distinction matters because explanation methods can be sensitive to model, explainer, and sampling choices [7], [8].

Applied XAI-IDS work increasingly evaluates explanation cost and agreement alongside importance plots. Nugraha *et al.* combine ANOVA and global SHAP selection with local SHAP–LIME cross-validation, reporting an 87% reduction in LIME generation time after feature reduction [27]. Their cross-explainer design addresses a different question from RF-reference/student agreement and supports treating explanation behavior as an evaluated subsystem.

D. Edge-IIoTset and Hardware-Aware Evaluation

Edge-IIoTset provides heterogeneous IoT/IIoT traffic and 15-class attack evaluation [28]. Hardware-aware studies report strong Edge-IIoTset classifiers at tens to hundreds of KB [29], and Gao *et al.* study lightweight temporal convolutional networks under explicit feature reduction [30]. Cross-paper comparisons remain sensitive to which CSV, attack labels,

identifier fields, payload fields, and ports are retained. We therefore treat Edge-IIoTset as a protocol-controlled robustness study and never merge its scores with the WSN-DS task.

The surrounding Edge-IIoT literature illustrates this protocol diversity. Seyedkolaei *et al.* report 0.946 F1 for a 15-class CNN [31]; Hasan *et al.* report 99.94% accuracy/F1 for a decision-tree pipeline and 0.187 ms multiclass inference on Jetson Nano [32]; and Alshehri *et al.* report 99.95% accuracy and 99.53% F1 after two-stage cleaning and self-attention DCNN modeling [33]. Badhan combines neuro-symbolic rules and KD, reporting 94.3% accuracy, 93.5% macro-F1, and relative memory/latency reductions [34]. Adjewa *et al.* instead offload transformer embeddings and deploy a 137 KB end-device classifier [35]. Differences in retained records, feature construction, averaging conventions, and execution boundaries preclude a common leaderboard.

Within the 65-paper corpus inspected for this study, executable model cores, cross-model explanation agreement, and controlled compression ablations appear in separate lines of work. CuKD-XAI integrates them through fixed-capacity students, route-level ablations, RF-reference/student attribution auditing, host-format conversion, full-test fixed-point replay on two MCU architectures, and a separately scoped Edge-IIoTset robustness study. The novelty claim is explicitly bounded to the inspected corpus.

III. SYSTEM OVERVIEW AND RESEARCH DESIGN

Fig. 1 separates model development from four independent audits. Export, integer equivalence, and live packet processing represent progressively broader claims, each requiring its own evidence.

Table I fixes the three experimental contracts before model-level results are introduced. The two Edge-IIoTset protocols are independent experiments rather than alternative preprocessing of one shared split.

A. WSN-DS Data and Split

The repository copy of WSN-DS contains 374,661 records and 19 columns. Removing the identifier and target leaves 17 numerical features. Class counts are 10,049 Blackhole, 3,312 Flooding, 14,596 Grayhole, 340,066 Normal, and 6,638 TDMA. A fixed stratified 70/15/15 split with random state 42 produces 262,252 training, 56,209 validation, and 56,200 test records. Ten model seeds vary RF randomness and neural initialization while the split remains fixed:

$$\{42, 123, 456, 789, 1001, 2024, 3141, 5678, 8192, 9999\}.$$

The archived experiment fits `StandardScaler` on the complete feature matrix before splitting. Test-set means and variances therefore influence the scaling transform. All multi-seed WSN predictive tables, SHAP audit, and the primary HIL fidelity table are consequently labeled as archived-protocol results. A train-only-scaler confirmation for seed-42 RF-KD students has since been completed (deployment training, host ONNX export, fixed-point export, and four full-test MCU replays). That confirmation closes the preprocessing-lineage gap for deployment evidence but is a single-seed artifact route, not a replacement ten-seed distribution.

B. Teachers and Compact Students

The primary tree teacher is an RF with 500 trees and maximum depth 15 [36]. For KD, its probabilities are isotonic-calibrated with three-fold cross-validation [37]. The neural teacher is a 17–128–256–128–5 MLP with batch normalization, ReLU, and dropout. It has 69,893 trainable parameters and a 273.02 KB FP32 parameter payload.

Student A is 17–32–16–5 (1,189 parameters, 1,136 MACs), and Student B is 17–64–32–5 (3,397 parameters, 3,296 MACs). ReLU follows both hidden layers; the output layer produces five logits. Fig. 2 shows what is transferred, what remains trainable, and which representations are evaluated after training. Both students are independently parameterized fixed-capacity networks.

C. Training Routes

For target y , student logits z_s , temperature T , and KD weight α , define $p_s^{(T)} = \sigma(z_s/T)$. A neural teacher with logits z_t supplies $q_t^{(T)} = \sigma(z_t/T)$. The calibrated RF instead supplies probabilities r , clipped below at 10^{-8} ; the implementation uses pseudo-logits $\log r$, so its softened target is

$$q_{\text{RF},i}^{(T)} = \frac{\exp(\log r_i/T)}{\sum_j \exp(\log r_j/T)} = \frac{r_i^{1/T}}{\sum_j r_j^{1/T}}. \quad (1)$$

With $q_t^{(T)}$ selected from the neural or RF definition, the single-teacher objective is

$$\mathcal{L}_{\text{KD}} = (1 - \alpha) \mathcal{L}_{\text{CE}}(z_s, y) + \alpha T^2 D_{\text{KL}}(q_t^{(T)} \| p_s^{(T)}). \quad (2)$$

A validation grid over $T \in \{2, 3, 4, 5\}$ and $\alpha \in \{0.5, 0.7, 0.9\}$ selects $T = 2$ and $\alpha = 0.5$. Inverse-frequency class weights are used in cross entropy. Standard neural runs use AdamW, learning rate 10^{-3} , weight decay 10^{-3} , cosine annealing, batch size 256, 30 epochs, and early-stopping patience eight.

The experiment matrix is summarized in Table II. Configurations C and F are archival aliases of their nominal-schedule variants and count once in statistical comparisons. The loss curriculum trains a three-epoch probe, orders examples by per-example loss, and exposes 33%, 66%, and 100% of the training set for 3, 3, and 24 nominal epochs. This is not compute matched to 30 full-data epochs because the first stages use subsets and global early stopping can terminate before the final stage. The extended route uses 5, 5, and 30 nominal epochs. Random pacing is the ordering control. The SMOTE route changes the neural teacher's training distribution using synthetic minority oversampling [38].

For co-distillation, let $\mathcal{K}_{\text{RF}} = T^2 D_{\text{KL}}(q_{\text{RF}}^{(T)} \| p_s^{(T)})$ and $\mathcal{K}_{\text{CL}} = T^2 D_{\text{KL}}(q_{\text{CL}}^{(T)} \| p_s^{(T)})$. Configuration J combines one hard-label term with the two distinct soft-target terms:

$$\mathcal{L}_J = 0.3 \mathcal{L}_{\text{CE}}(z_s, y) + 0.4 \mathcal{K}_{\text{RF}} + 0.3 \mathcal{K}_{\text{CL}}. \quad (3)$$

Configuration J uses $T = 2$, learning rate 7×10^{-4} , 40 epochs, and patience ten. Results are reported with this larger compute budget.

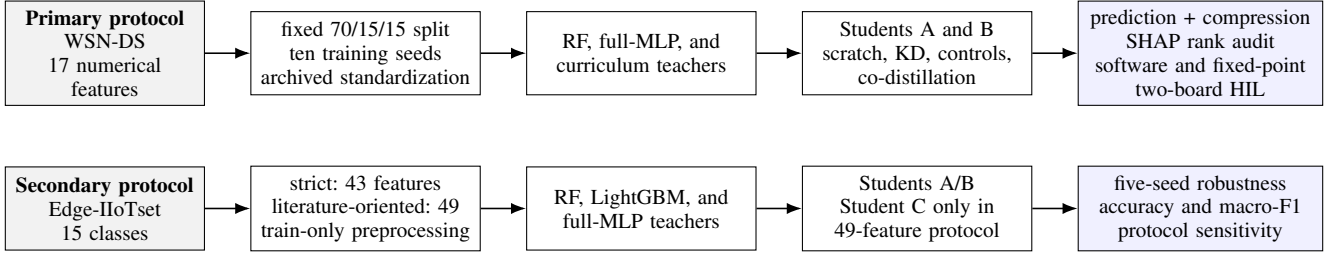


Fig. 1. Research design. The WSN-DS branch provides the primary compression, explanation, conversion, and HIL evidence. Edge-IIoTset supplies a separately reported robustness study with different data construction.

TABLE I
EXPERIMENTAL PROTOCOLS

Protocol / source	Rows	Classes	Features	Seeds	Split	Preprocessing contract
WSN-DS / WSN-DS CSV	374,661	5	17	10	fixed stratified 70/15/15	Identifier and target removed; archived <code>StandardScaler</code> fitted on the complete feature matrix before splitting.
Edge strict / ML-EdgeIoT CSV	157,800	15	43	5	fixed stratified 70/15/15	Identifiers, source/payload fields, ports, auxiliary targets, all-missing, and constant columns removed; training-only imputation, categorical encoding, and scaling.
Edge literature-oriented / DNN-EdgeIoT CSV	2,219,201	15	49	5	fixed stratified 70/15/15	Thirteen identifier/source/payload/leakage fields removed while ports are retained; training-only imputation, categorical encoding, and scaling.

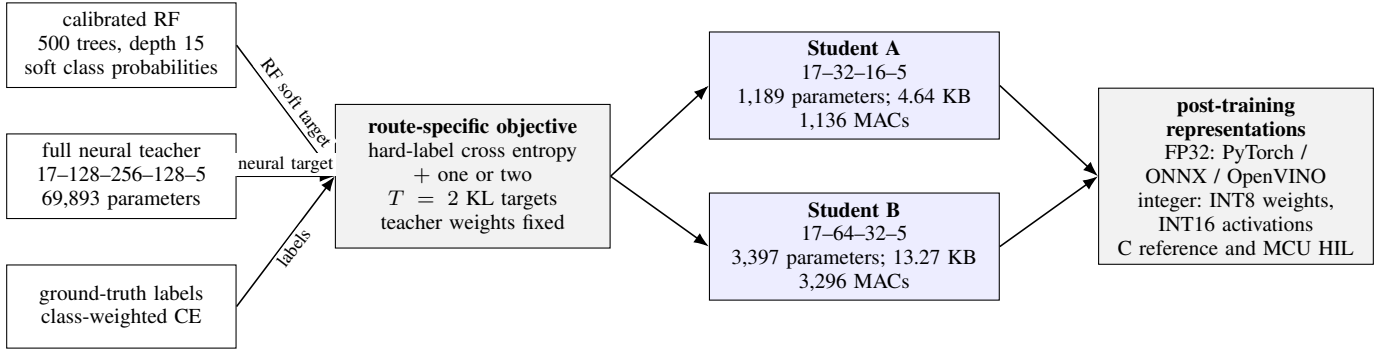


Fig. 2. Compression and deployment structure for WSN-DS. Training routes select one or two teacher targets together with the hard-label term; the student architecture itself is unchanged. Both students are evaluated in FP32/software representations, while the RF-distilled instances are additionally converted to fixed-point C and replayed on both boards. Parameter counts include weights and biases.

TABLE II
WSN-DS EXPERIMENT ROUTES

Code	Model trained	Supervision / schedule
A	RF-500	hard labels
B	full MLP	hard labels
C2	full MLP	domain curriculum
C	full MLP	nominal-schedule loss curriculum (alias)
C-ext	full MLP	extended loss curriculum
C-fair	full MLP	nominal-schedule loss curriculum
D	compact student	hard labels
E2	compact student	full-MLP KD
E	compact student	calibrated RF-KD
F	compact student	nominal-schedule curriculum-KD (alias)
F-ext	compact student	extended curriculum-KD
F-fair	compact student	nominal-schedule curriculum-KD
G	compact student	random-pacing teacher KD
I	compact student	SMOTE-trained MLP KD
J	compact student	RF + curriculum co-distillation

D. Explanation Audit

TreeExplainer produces RF attributions and DeepExplainer produces Student A attributions [26]. The preserved audit uses 100 background samples and 500 explained samples. Its student is the final-seed Student A distilled from the nominal-schedule curriculum MLP (the historical F-fair key), not the RF-KD or co-distilled model; this is the only preserved model pair with SHAP evidence. Global importance is the mean absolute SHAP value by feature. Spearman correlation compares the 17 feature ranks. Five repeated subsamples provide descriptive sensitivity through their mean and standard deviation.

E. Software and Fixed-Point Deployment

The deployment path has four numerical representations:

- 1) PyTorch FP32 and quantization-aware training (QAT) artifacts [39];
- 2) FP32 and dynamic-INT8 ONNX graphs executed by ONNX Runtime [40], [41];

- 3) FP32 ONNX graphs imported by OpenVINO [42]; and
- 4) generated C with integer StandardScaler preprocessing, INT8 weights, INT32 biases/accumulators, and INT16 layer activations.

Host-runtime measurements use one preserved artifact per route on the recorded Windows 10/Python 3.11.9 environment. Each ONNX Runtime and OpenVINO latency distribution uses 50 warm-up calls followed by 1,000 batch-one and 300 batch-64 timed calls; Table IX reports the batch-one median and 95th percentile. The archived environment omits the host CPU model, limiting these values to within-machine format comparisons.

The HIL host replays already extracted 17-feature records over USB serial. Firmware performs integer preprocessing and dense inference, then returns status, prediction, and timing. Both RF-KD artifacts originate from the recorded proof run with model seed 9999. The data split uses random state 42, and the exporter records test-vector seed 42; at the full replay count, selection resolves to all ordered test indices 0–56,199. Board timing excludes host serialization and covers on-device preprocessing plus inference. Therefore, HIL establishes fixed-point execution of the tabular detector, not radio capture, packet-to-feature extraction, or end-to-end network throughput.

F. Edge-IIoTset Protocols

Both Edge-IIoTset studies use seeds {42, 123, 456, 789, 1001}, fixed stratified 70/15/15 splits, and train-only preprocessing. Categorical vocabularies, imputation values, and scaling statistics are learned from the training partition; categorical levels occurring fewer than ten times are grouped and encoded cardinality is capped at 64. Neural training uses the same 30-epoch, batch-256, AdamW schedule as the WSN study. RF teachers use 500 trees, depth 15, and three-fold isotonic calibration. The WSN-selected $T = 2$ and $\alpha = 0.5$ are transferred without Edge-IIoTset retuning. The literature-oriented protocol additionally evaluates LightGBM with 500 estimators, learning rate 0.05, and 31 leaves.

The *strict* protocol uses the 157,800-row ML-EdgeIoT file. It removes 16 identifier, source, payload, port, and label-leakage fields, followed by all-missing and constant columns, yielding 43 features and 15 classes. It evaluates Students A and B, whose parameter counts increase to 2,191 and 5,391 because of the larger input and output dimensions.

The *literature-oriented* protocol uses the 2,219,201-row DNN-EdgeIoT file. It removes 13 identifier/source/payload/leakage fields but retains port fields, yielding 49 features and 15 classes. Students A, B, and C use hidden widths 32–16, 64–32, and 128–64. The protocol name indicates closer alignment with common Edge-IIoT studies; exact cross-paper equivalence still depends on retained fields and preprocessing.

IV. EXPERIMENTAL EVALUATION

A. Metrics and Statistical Scope

Accuracy is reported for continuity with prior work. Macro-F1 is primary because it weights each class equally. Per-class

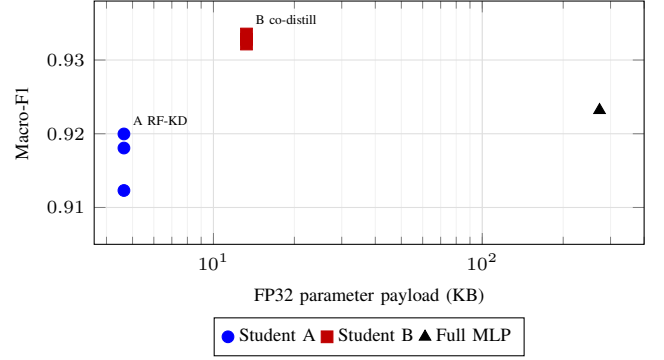


Fig. 3. WSN-DS size–performance trade-off. Points are ten-seed means; all configurations at a student capacity share the same FP32 parameter size.

F1 exposes the Blackhole/Grayhole trade-off obscured by the dominant Normal class. Ten-seed values are mean $\pm$ population standard deviation as stored by the experiment code. Paired, two-sided Wilcoxon signed-rank tests compare seed-level macro-F1 values [43]. For the ten unique co-distillation comparisons archived across two students, Holm’s step-down procedure controls family-wise error; raw and adjusted p values are both reported. These tests remain exploratory because $n = 10$ and all seeds share one split.

Latency tables evaluate one preserved artifact per route on its recorded host environment, whereas predictive tables summarize multi-seed training. HIL metrics cover every test record. Model storage is always labeled by basis: serialized pickle, raw FP32 parameter payload, serialized ONNX/PyTorch artifact, fixed-point model-only constants, or framework-linked firmware total.

B. RQ1: Predictive Performance and Compression

Table III gives the central ten-seed result. The RF teacher is strongest at 0.97889 ± 0.00025 macro-F1. The 69,893-parameter full MLP reaches 0.92319 ± 0.00238 . Student A RF-KD reaches 0.91997 ± 0.00312 in 4.64 KB, 0.00767 above Student A scratch. Student B is stronger by capacity: scratch already reaches 0.93217, RF-KD reaches 0.93281, and co-distillation reaches 0.93353.

Table IV shows why aggregate accuracy is insufficient. Student A RF-KD is stronger than Student A co-distillation on Blackhole and Grayhole. Student B co-distillation has the best Student B Blackhole F1 (0.878) but RF-KD has stronger Grayhole F1 (0.902 versus 0.891). The Normal class remains near 0.998 for all selected students.

C. Compression Accounting

The commonly quoted RF-to-student ratios compare an 85,064.54 KB serialized RF pickle with raw FP32 neural parameters. Under that mixed but explicit basis, Student A and B are $18,315\times$ and $6,411\times$ smaller. The same-basis neural comparison is cleaner: relative to the full MLP’s FP32 parameter payload, Students A and B are $58.8\times$ and $20.6\times$ smaller. ONNX serialization produces 5.44 and 14.07 KB artifacts. Fixed-point model constants require 1,348 and 3,700 bytes,

TABLE III
SELECTED WSN-DS TEN-SEED RESULTS. NEURAL SIZES ARE RAW FP32 PARAMETER PAYLOADS; RF SIZE IS THE RECORDED MEAN
SERIALIZED-PICKLE SIZE ACROSS THE TEN RUNS.

Model	Route	Parameters	Size (KB)	Accuracy	Macro-F1	$\Delta F1$ vs scratch
RF teacher	supervised	–	85,064.54 [†]	0.99660 ± 0.00005	0.97889 ± 0.00025	–
Full MLP	supervised	69,893	273.02	0.98732 ± 0.00040	0.92319 ± 0.00238	–
Student A	scratch	1,189	4.64	0.98471 ± 0.00213	0.91230 ± 0.00622	–
Student A	MLP-KD	1,189	4.64	0.98613 ± 0.00043	0.91635 ± 0.00276	+0.00405
Student A	RF-KD	1,189	4.64	0.98688 ± 0.00052	0.91997 ± 0.00312	+0.00767
Student A	co-distill	1,189	4.64	0.98651 ± 0.00070	0.91806 ± 0.00410	+0.00576
Student B	scratch	3,397	13.27	0.98883 ± 0.00064	0.93217 ± 0.00375	–
Student B	MLP-KD	3,397	13.27	0.98807 ± 0.00108	0.92755 ± 0.00682	–0.00462
Student B	RF-KD	3,397	13.27	0.98911 ± 0.00125	0.93281 ± 0.00759	+0.00064
Student B	co-distill	3,397	13.27	0.98913 ± 0.00185	0.93353 ± 0.01136	+0.00136

[†]Serialized RF artifact; its storage representation differs from raw neural parameters.

TABLE IV
SELECTED WSN-DS MEAN PER-CLASS F1

Model	BH	Flood	Gray	Normal	TDMA	Macro
RF	0.990	0.963	0.983	0.998	0.959	0.979
Full MLP	0.869	0.939	0.877	0.997	0.935	0.923
A RF-KD	0.838	0.953	0.859	0.998	0.952	0.920
A co-distill	0.837	0.953	0.856	0.998	0.947	0.918
B scratch	0.891	0.941	0.898	0.997	0.933	0.932
B RF-KD	0.873	0.953	0.889	0.998	0.951	0.933
B co-distill	0.878	0.952	0.891	0.998	0.948	0.934

TABLE V
STORAGE MEASUREMENTS AND THEIR MEANING

Artifact	Size	Measurement basis
RF teacher	85,064.54 KB	serialized pickle
Full MLP	273.02 KB	raw FP32 parameters
Student A	4.64 KB	raw FP32 parameters
Student B	13.27 KB	raw FP32 parameters
Student A ONNX	5.44 KB	serialized FP32 graph
Student B ONNX	14.07 KB	serialized FP32 graph
Student A fixed core	1,348 B	weights + biases only
Student B fixed core	3,700 B	weights + biases only

excluding code, preprocessing constants, buffers, protocol, and board framework.

D. RQ2: Effects of Distillation and Curriculum

The complete route matrices appear in Appendix A. Three conclusions follow.

First, RF-KD is the most reliable Student A route. Its mean macro-F1 is 0.91997, compared with 0.91635 for MLP-KD and 0.91291 for nominal-schedule curriculum-KD. Second, Student B gains little from KD because scratch training is already strong. MLP-KD lowers Student B macro-F1 by 0.00462, whereas RF-KD and co-distillation improve it by only 0.00064 and 0.00136. Third, longer curriculum training is unstable: the extended full MLP has 0.86726 ± 0.15482 macro-F1, and extended curriculum-KD has standard deviations near 0.025 for both students. The additional nominal epochs increase variance without a reliable gain.

Table VI reports the paired co-distillation comparisons. After Holm adjustment, Student A co-distillation is better than nominal-schedule curriculum-KD (mean difference 0.0051,

TABLE VI
CO-DISTILLATION PAIRED WILCOXON TESTS ON WSN-DS MACRO-F1.
HOLM ADJUSTMENT COVERS THE TEN UNIQUE COMPARISONS SHOWN.

Student	Comparator	Mean diff.	W	p_{raw}	p_{Holm}
A	Scratch	0.0058	11.0	0.1055	0.6328
A	RF-KD	-0.0019	20.0	0.4922	1.0000
A	MLP-KD	0.0017	15.0	0.2324	0.9668
A	Curriculum-KD (nominal schedule)	0.0051	1.0	0.0039	0.0352
A	Full MLP	-0.0051	0.0	0.0020	0.0195
B	Scratch	0.0014	23.0	0.6953	1.0000
B	RF-KD	0.0007	27.0	1.0000	1.0000
B	MLP-KD	0.0060	14.0	0.1934	0.9668
B	Curriculum-KD (nominal schedule)	0.0077	8.0	0.0488	0.3418
B	Full MLP	0.0103	3.0	0.0098	0.0781

$p_{Holm} = 0.0352$) but worse than the full MLP (mean difference -0.0051 , $p_{Holm} = 0.0195$). Its differences from Student A scratch, RF-KD, and MLP-KD are not significant. No Student B comparison survives adjustment: in particular, the 0.0014 mean gain over scratch and 0.0007 gain over RF-KD are unsupported. The tests therefore reject universal co-distillation superiority and show that the apparent Student B advantages in raw p values are not robust to family-wise correction.

E. RQ3: Explanation Fidelity

The audited Student A curriculum-KD model and RF reference have global rank correlation $\rho = 0.0466$ with $p = 0.8591$. Across five repeated subsamples, mean ρ is 0.0015 with standard deviation 0.1068. Per-class correlations are 0.211 (Blackhole), 0.409 (Flooding), -0.015 (Grayhole), -0.081 (Normal), and -0.150 (TDMA); none is significant in the preserved output. Because this student was distilled from the curriculum MLP rather than the RF, the comparison measures cross-model alignment and cannot establish attribution transfer from the actual distillation teacher.

The mismatch extends across the ranking. The student’s five leading features are `Is_CH`, `Time`, `who CH`, `dist_CH_To_BS`, and `Data_Sent_To_BS`. The RF’s five are `ADV_S`, `Is_CH`, `Data_Sent_To_BS`, `DATA_S`, and `Expanded Energy`. Table VII provides all 17 ranks. The result answers RQ3 negatively for this model pair; Student

TABLE VII
GLOBAL SHAP FEATURE RANKS FOR THE AUDITED PAIR

Feature	RF rank	Student rank	Δ rank
Is_CH	2	1	-1
Time	16	2	-14
who_CH	15	3	-12
dist_CH_To_BS	9	4	-5
Data_Sent_To_BS	3	5	2
ADV_R	12	6	-6
JOIN_S	10	7	-3
DATA_R	13	8	-5
SCH_S	7	9	2
DATA_S	4	10	6
ADV_S	1	11	10
Dist_To_CH	11	12	1
Rank	8	13	5
Expanded Energy	5	14	9
send_code	6	15	9
SCH_R	17	16	-1
JOIN_R	14	17	3

TABLE VIII
SINGLE-ARTIFACT PyTorch QAT RESULTS

Model	Accuracy	Macro-F1	Δ vs FP32
D A scratch	0.9823	0.9003	-0.0067
E A KD from RF	0.9812	0.8996	-0.0178
J A CoDistill RF CL	0.9854	0.9212	-0.0041
D B scratch	0.9844	0.9085	-0.0217
E B KD from RF	0.9869	0.9213	-0.0234
J B CoDistill RF CL	0.9853	0.9121	-0.0197

B, RF-KD, and co-distillation lack corresponding attribution audits.

F. RQ4: Software Runtime and Quantization

1) *PyTorch QAT*: QAT was evaluated for D, E, and J at one proof seed (9999) on the recorded Windows/PyTorch environment. Table VIII shows that all six QAT artifacts lose macro-F1 relative to their corresponding FP32 artifacts. The smallest drop is 0.00414 for Student A co-distillation; the largest is 0.02336 for Student B RF-KD. These runs establish convertibility with measured, model-dependent loss.

2) *ONNX Runtime and OpenVINO*: FP32 ONNX batch-one median latency is 0.0267–0.0289 ms for the six compact artifacts. OpenVINO produces identical predictions to ONNX in every evaluated pair, but median latency is about 0.129–0.132 ms in this environment. Dynamic INT8 ONNX reduces Student B graph size from 14.07 to 7.37 KB, but lowers macro-F1 and has a slower batch-one median (approximately 0.039–0.040 ms) than ONNX FP32. Table IX reports all 18 runtime rows; on this host, neither lower precision nor OpenVINO improves latency.

3) *Fixed-Point HIL Fidelity*: Table X is the strongest executable evidence. All four model-board pairs completed 56,200 vectors with no missing, duplicate, unexpected, or non-OK responses. MCU predictions agree 100% with generated fixed-point references. Agreement with FP32 is 0.9950 for Student A and 0.9939 for Student B, so remaining differences arise from fixed-point conversion rather than serial transport or board-specific execution.

Student A is faster because it performs 1,136 rather than 3,296 MACs. Table XI normalizes timing. ESP32-C3 requires

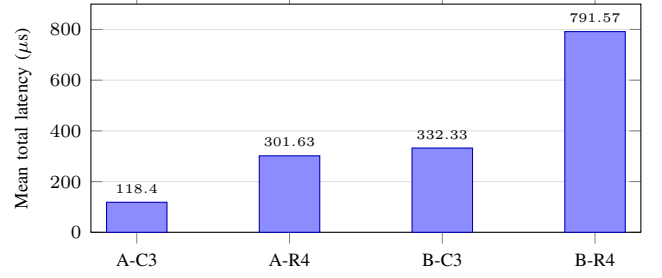


Fig. 4. Measured fixed-point preprocessing-plus-inference latency over all 56,200 records. C3 denotes ESP32-C3 and R4 denotes Arduino UNO R4 WiFi.

approximately 15.8 cycles/MAC for either model, while R4 requires 11.2–11.8. The compute-only processing ceilings range from 1,263 to 8,446 vectors/s. These are reciprocals of on-device preprocessing-plus-inference time, not serial or radio throughput.

Fixed-point drift affects 281 Student A records (0.500%) and 343 Student B records (0.610%). Student A drift is concentrated in Blackhole (107) and Grayhole (150), with 23 Normal and one TDMA record. Student B drift contains 261 Blackhole, 65 Grayhole, 16 Normal, and one TDMA record. Neither model changes a Flooding prediction relative to FP32. Thus a single agreement percentage hides class-specific numerical sensitivity.

4) *Train-Only Scaler Deployment Confirmation*: To address the pre-split scaling limitation for executable claims, RF-KD Students A and B were retrained at seed 42 with a train-fitted `StandardScaler` on the same archived split indices. Host ONNX Runtime evaluation of the train-only FP32 graphs matches PyTorch on every test record (agreement 1.0); OpenVINO CPU execution of the same ONNX graphs also agrees exactly with PyTorch and ONNX Runtime (agreement 1.0) with batch-one median latency about 0.110–0.114 ms on the evaluation host. Dynamic INT8 ONNX reduces macro-F1 (Student A 0.9485 $\rightarrow$ 0.8938, Student B 0.9449 $\rightarrow$ 0.9066) and is retained only as a software-size probe. Direct post-training fixed-point conversion of the train-only FP32 weights was selected for MCU export after a QAT refine probe lowered absolute Student A macro-F1. On ESP32-C3 and Arduino UNO R4 WiFi, all four train-only model-board pairs completed 56,200 serial replays with 100% MCU-to-fixed-reference agreement and all-OK status; MCU-to-FP32 agreement is 0.9919 (A) and 0.9905 (B), matching the export conversion gap. Board mean total latency is 116.5 μ s (ESP32-C3 Student A), 320.3 μ s (ESP32-C3 Student B), 301.5 μ s (R4 Student A), and 791.4 μ s (R4 Student B). Fixed MCU macro-F1 is 0.9244 (A) and 0.9180 (B). Arduino CLI recompiles on the HIL host record flash footprints of 281.8–284.1 KB (ESP32-C3) and 56.4–58.7 KB (R4) with ID-bound binaries; post-flash 10-row smokes on all four pairs returned 10/10 OK. Artifacts live under copy-suffixed paths so archived tables remain unaltered.

A separate five-seed exact-feature-group sensitivity package (seeds {42, 123, 456, 789, 1001}, train-only scaler, partitions disjoint on identical feature vectors) yields RF-KD mean

TABLE IX
SINGLE-ARTIFACT ONNX/OPENVINO RUNTIME RESULTS

Model	Runtime	Accuracy	Macro-F1	Size KB	p50 ms	p95 ms	Agreement
D A scratch	ONNX FP32	0.9846	0.9069	5.44	0.0267	0.0336	–
D A scratch	ONNX dynamic INT8	0.9812	0.8914	5.07	0.0393	0.0513	–
D A scratch	OpenVINO FP32	0.9846	0.9069	5.44	0.1289	0.2800	1.000
E A KD from RF	ONNX FP32	0.9864	0.9175	5.44	0.0275	0.0689	–
E A KD from RF	ONNX dynamic INT8	0.9837	0.9006	5.07	0.0395	0.1013	–
E A KD from RF	OpenVINO FP32	0.9864	0.9175	5.44	0.1307	0.4292	1.000
J A CoDistill RF CL	ONNX FP32	0.9878	0.9254	5.44	0.0279	0.0850	–
J A CoDistill RF CL	ONNX dynamic INT8	0.9832	0.8993	5.07	0.0395	0.0602	–
J A CoDistill RF CL	OpenVINO FP32	0.9878	0.9254	5.44	0.1318	0.4457	1.000
D B scratch	ONNX FP32	0.9885	0.9302	14.07	0.0289	0.0308	–
D B scratch	ONNX dynamic INT8	0.9841	0.9041	7.37	0.0392	0.0544	–
D B scratch	OpenVINO FP32	0.9885	0.9302	14.07	0.1313	0.4624	1.000
E B KD from RF	ONNX FP32	0.9910	0.9447	14.07	0.0285	0.0713	–
E B KD from RF	ONNX dynamic INT8	0.9875	0.9231	7.37	0.0396	0.0549	–
E B KD from RF	OpenVINO FP32	0.9910	0.9447	14.07	0.1304	0.4743	1.000
J B CoDistill RF CL	ONNX FP32	0.9889	0.9318	14.07	0.0284	0.0594	–
J B CoDistill RF CL	ONNX dynamic INT8	0.9872	0.9213	7.37	0.0401	0.1055	–
J B CoDistill RF CL	OpenVINO FP32	0.9889	0.9318	14.07	0.1302	0.2785	1.000

TABLE X
FULL-TEST-SET FIXED-POINT HIL RESULTS

Model	Board	Vectors	Macro-F1	MCU/fixed	MCU/FP32	Mean total μ s	p99 μ s
Student A RF-KD	ESP32-C3 DevKitM-1	56,200	0.9140	1.000	0.9950	118.40	125
Student A RF-KD	Arduino R4 WiFi	56,200	0.9140	1.000	0.9950	301.63	305
Student B RF-KD	ESP32-C3 DevKitM-1	56,200	0.9181	1.000	0.9939	332.33	338
Student B RF-KD	Arduino R4 WiFi	56,200	0.9181	1.000	0.9939	791.57	795

TABLE XI
HIL COMPUTE NORMALIZATION

Model	Board	MACs	Inf. μ s	Cycles	cyc/MAC	vec/s
Student A RF-KD	ESP32-C3	1,136	112.31	17970	15.82	8445.7
Student A RF-KD	Arduino R4	1,136	280.36	13457	11.85	3315.3
Student B RF-KD	ESP32-C3	3,296	325.08	52013	15.78	3009.1
Student B RF-KD	Arduino R4	3,296	770.68	36993	11.22	1263.3

TABLE XII
FIXED-POINT MODEL CORE AND PREDICTION DRIFT

Model	Architecture	MACs	Param B	Drift	Drift %
Student A RF-KD	17-32-16-5	1,136	1,348	281	0.500%
Student B RF-KD	17-64-32-5	3,296	3,700	343	0.610%

macro-F1 0.9141 ± 0.0069 (Student A) and 0.9281 ± 0.0074 (Student B). Mean paired KD-minus-scratch macro-F1 deltas are near zero ($+0.0002$ for A, -0.0017 for B). That package is a descriptive leakage-control sensitivity analysis, not a matched causal ablation of the archived random-row route and not a significance claim.

5) *Framework-Linked Compile Footprint*: Arduino compile totals include board support and serial replay code. To isolate the incremental payload, Table XIII subtracts a serial-only baseline compiled for the same board environment. Program deltas are 4,248–7,036 bytes and global-variable deltas are 388–392 bytes. These deltas are more informative for model integration than presenting the 278–281 KB ESP32 framework total as model size.

6) *MSP430F1611 Memory Feasibility*: The Student A preprocessing and inference core was cross-compiled for

TABLE XIII
BOARD COMPILATION TOTALS AND BASELINE DELTAS

Model	Board	Prog. B	Glob. B	Δ prog.	Δ glob.
A RF-KD	C3	278,836	13,556	4,680	392
A RF-KD	R4	56,104	7,128	4,248	388
B RF-KD	C3	281,192	13,556	7,036	392
B RF-KD	R4	58,440	7,128	6,584	388

MSP430F1611 using MSP430-GCC 9.3.1.11 with `-Os`. The linked smoke firmware reports 2,842 bytes of text-class storage, zero data bytes, six BSS/heap bytes, and a conservative project-function stack estimate of about 248 bytes during prediction. The inference object contributes 494 B `.text` and 1,348 B `.rodata`; preprocessing contributes 412 B `.text` and 136 B `.rodata`. This supports memory feasibility against the nominal 48 KB flash and 10 KB RAM of the target class. Wider arithmetic invokes MSP430 helper routines, so static size alone cannot predict cycle latency. Physical TelosB execution remains future work.

G. RQ5: Edge-IIoTset Robustness

Table XIV summarizes the two protocols. Under strict feature removal, the RF teacher reaches 0.8729 macro-F1, whereas the best compact model is Student B RF-KD at 0.7111. Co-distillation is nearly tied at 0.7100. Under the literature-oriented protocol, RF reaches 0.8892, LightGBM reaches 0.8861, and Student C RF-KD reaches 0.8244 in 61.06 KB. Student C scratch is only 0.00047 lower, and Student B LightGBM-KD is 0.00302 above Student B scratch. KD benefits are therefore small and architecture-dependent.

TABLE XIV
SELECTED EDGE-IIOTSET FIVE-SEED RESULTS

Protocol	Model	Route	Parameters	Size KB	Accuracy	Macro-F1
Strict (43 feat.)	RF	supervised	–	109,879.57	0.8833 ± 0.0002	0.8729 ± 0.0002
Strict (43 feat.)	Student A	scratch	2,191	8.56	0.7001 ± 0.0191	0.6646 ± 0.0200
Strict (43 feat.)	Student A	co-distill	2,191	8.56	0.7243 ± 0.0057	0.7010 ± 0.0065
Strict (43 feat.)	Student B	scratch	5,391	21.06	0.7236 ± 0.0011	0.6878 ± 0.0018
Strict (43 feat.)	Student B	RF-KD	5,391	21.06	0.7309 ± 0.0031	0.7111 ± 0.0025
Literature (49 feat.)	RF	supervised	–	101,114.94	0.9825 ± 0.0003	0.8892 ± 0.0011
Literature (49 feat.)	LightGBM	supervised	–	15,224.05	0.9867 ± 0.0001	0.8861 ± 0.0003
Literature (49 feat.)	Student A	LightGBM-KD	2,383	9.31	0.9661 ± 0.0013	0.8143 ± 0.0039
Literature (49 feat.)	Student B	LightGBM-KD	5,775	22.56	0.9680 ± 0.0002	0.8220 ± 0.0010
Literature (49 feat.)	Student C	scratch	15,631	61.06	0.9687 ± 0.0001	0.8239 ± 0.0003
Literature (49 feat.)	Student C	RF-KD	15,631	61.06	0.9685 ± 0.0002	0.8244 ± 0.0009

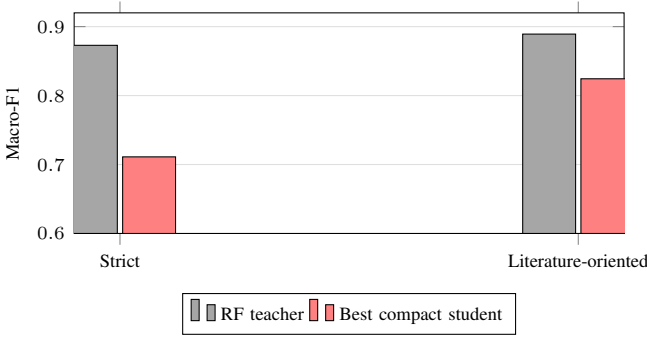


Fig. 5. Protocol sensitivity on Edge-IIoTset. The protocols use different source files and retained features, so the gap is not attributed to training alone.

The 0.1133 macro-F1 difference between the best compact strict and literature-oriented results arises alongside changes in source file, retained features, and input distribution. This protocol sensitivity is itself a robustness finding and cautions against comparing Edge-IIoTset scores across different identifier, payload-field, or port-retention policies.

V. COMPARISON WITH PRIOR WORK

Tables XV and XVI separate direct WSN-DS comparisons from cross-dataset compression and deployment studies. Metric labels retain each source’s terminology, including unspecified F1 averaging. CuKD-XAI trails the strongest WSN-DS ensembles in raw performance. Its differentiating evidence is the combination of KB-scale dense students, full-test-set fixed-point execution, and a quantitative RF-reference/student explanation audit.

Two comparisons should not be collapsed. First, the $18,315\times$ RF-to-Student-A ratio is larger than ratios reported by many neural compression papers partly because a Python RF pickle and raw neural weights have different serialization overhead. Second, sub-millisecond HIL model-core timing cannot be compared with a gateway window that includes a different feature window and system boundary. The table is therefore positioning evidence, not a universal leaderboard.

VI. DISCUSSION

A. Answers to the Research Questions

RQ1: Useful minority-sensitive performance survives large compression, but the RF gap remains. Student A RF-KD loses 0.0589 macro-F1 relative to RF while reducing the stated artifact basis from 85,064.54 KB to 4.64 KB. Student B co-distillation loses 0.0454 and uses 13.27 KB. Relative to the full MLP on the same FP32 parameter basis, Student B is $20.6\times$ smaller and 0.0103 higher in mean macro-F1.

RQ2: Teacher choice matters more than curriculum complexity. RF-KD is best by mean macro-F1 for Student A; Student B scratch, RF-KD, and co-distillation are effectively close. Curriculum-only teachers do not improve either capacity, and extended curricula are unstable. After Holm correction, Student A co-distillation improves on nominal-schedule curriculum-KD but remains below the full MLP, while no Student B comparison is significant.

RQ3: The audited student does not align with the RF reference’s global SHAP ranking. Predictive utility and cross-model explanation similarity must therefore be reported independently. This finding is specific to one final-seed Student A curriculum-KD/RF-reference pair and motivates a matched-teacher, multi-seed explanation study.

RQ4: The selected RF-KD students are executable with exact generated-integer-reference fidelity on both boards. Quantization introduces 0.50–0.61% prediction drift relative to FP32, concentrated in Blackhole and Grayhole. Dynamic INT8 ONNX and QAT both introduce macro-F1 loss, and dynamic INT8 is slower than FP32 ONNX on the recorded host.

RQ5: Compact students transfer to Edge-IIoTset, but protocol and capacity dominate the result. The best compact macro-F1 ranges from 0.7111 to 0.8244 across the two protocols, and the modest KD gains indicate dataset-specific behavior.

B. What the Hardware Results Establish

The HIL experiment is stronger than a desktop C self-test because the same integer kernels execute on two different MCU architectures and reproduce 56,200 generated references exactly. Compile deltas and fixed-point constants also show that the model core is small relative to board frameworks. The

TABLE XV
POSITIONING AGAINST CLOSELY RELATED WORK. VALUES USE EACH PAPER’S PROTOCOL AND ARE NOT ASSUMED DIRECTLY COMPARABLE.

Work	Dataset / task	Reported predictive result	Reported deployment evidence	Relation to CuKD-XAI
Almomani <i>et al.</i> [9]	WSN-DS, 5 class	dataset baselines	none	Defines the primary task and features.
Alqahtani <i>et al.</i> [11]	WSN-DS, 5 class, 10-fold	detection rate 92.9–99.9% by class	1.905 s classification time for their test set	Strong class-sensitive tree baseline; no model-size or device result.
Salmi and Oughdir [12]	WSN-DS, 5 class	0.944 accuracy; 0.959 precision; 0.922 recall	absolute size/latency not reported	Earlier CNN–LSTM baseline under a different split/training protocol.
MLSTL-WSN [2]	WSN-DS, multiclass	99.92% accuracy	46% modeling-time reduction; absolute artifact size not reported	Stronger accuracy; uses SHAP for selection, not cross-model explanation agreement.
Talukder <i>et al.</i> [3]	WSN-DS, multiclass	99.94% accuracy and reported F1	absolute size/latency not reported	Stronger predictive result using balancing, PCA, and RF.
Birahim <i>et al.</i> [4]	WSN-DS, multiclass	99.73% accuracy; 99.72% reported F1	absolute size/latency not reported	Stronger ensemble accuracy and SHAP/LIME; no compressed neural firmware path.
Nguyen <i>et al.</i> [10]	WSN-DS, multiclass	99.62% accuracy in the primary report	absolute size/latency not reported	Optimized CatBoost is predictively stronger; model-core deployment is not evaluated.
Xiao and Duan [13]	WSN-DS, multiclass	95.62% accuracy; 96.07% reported F1	approximately 125 MB memory and 175 min training in the local paper	CuKD-XAI accuracy is higher and artifacts are much smaller; F1 averaging differs/unspecified.
Alfarra and AbuSamra [14]	WSN-DS hybrid 4-class gateway path	98% accuracy; 0.93 macro-F1	pruned INT8 CNN–LSTM, 42 ms and 28 mJ/window, lifetime simulation	Closest energy-aware comparator; their energy evidence is stronger, ours covers five-class MCU model-core execution.
CuKD-XAI (archived protocol)	WSN-DS, 5 class, fixed split	Student A: 0.98688 accuracy, 0.91997 macro-F1; Student B: 0.98913, 0.93353	4.64/13.27 KB FP32 parameters; four full-test HIL replays	Lower predictive ceiling; substantially stronger numerical and execution trace, subject to the disclosed scaling limitation.

TABLE XVI
CROSS-DATASET COMPRESSION, EXPLAINABILITY, AND DEPLOYMENT CONTEXT. REPORTED VALUES RETAIN EACH SOURCE’S TASK AND MEASUREMENT BOUNDARY.

Work	Dataset / task	Reported predictive result	Reported efficiency / deployment evidence	Relevance and boundary
Benaddi <i>et al.</i> [17]	TON_IoT, 9 class	0.9968 accuracy; 0.9863 macro-F1	3,042 parameters, 22.29 KB, 1.29 ms	Closest compact KD+SHAP design; different dataset and no fixed-point HIL.
Wisnwanichthan and Thammawichai [18]	five IoT/UAV IDS datasets	up to 6.12% improvement reported with KD	92–95% parameter reduction; 7–11% faster inference	Broad KD evidence; reductions are relative and dataset-specific.
Javed <i>et al.</i> [25]	smart-thermostat IDS	device-specific binary/multiclass results	quantized CatBoost/XGBoost executed on ESP32	Stronger application-integrated device study; different traffic and features.
Diab <i>et al.</i> [29]	Edge-IIoTset, 15 class	95.25% LightGBM; 96.73% HW-NAS CNN accuracy	74.93 KB/1.20K ops; 190.34 KB/838.89K FLOPs	Hardware-aware baseline; retained features and F1 definition differ.
Gao <i>et al.</i> [30]	DNN-EdgeIIoT, 15 class	93.79–94.24% accuracy	16.25–25.37 KB TCN	Compact Edge-IIoT comparison; no common preprocessing contract.
Hasan <i>et al.</i> [32]	Edge-IIoTset, binary/15 class	99.94% accuracy and reported F1	0.185/0.187 ms binary/multiclass inference on Jetson Nano	Physical edge timing; tree pipeline and protocol differ.
Badhan [34]	Edge-IIoTset	94.3% accuracy; 93.5% macro-F1	37% memory and 54% latency reduction vs. stated baselines	KD plus symbolic explanations; reductions are relative, not artifact bytes.
Adjewa <i>et al.</i> [35]	transformer-assisted IoT IDS	99.9% accuracy	137 KB end-device network; under 70 ms on CPU	Offloaded-embedding architecture; system boundary differs from local tabular inference.
Nugraha <i>et al.</i> [27]	three IDS traces	at least 99% reported F1	LIME time reduced from 36 to 4.9 s after feature reduction	Stronger cross-explainer/runtime study; no compact-model comparison or MCU result.

MSP430 cross-compile extends memory feasibility to a WSN-mote-class target. The train-only seed-42 four-pair matrix repeats that full-test fidelity under train-fitted scaling, confirming that exact MCU-to-fixed agreement is not an artifact of the archived pre-split scaler alone.

The experiment still begins with extracted tabular records. It does not implement a LEACH monitor, derive WSN-DS features from packets, run TinyOS/Contiki/RIOT, measure radio traffic, or quantify energy. Those are system-integration questions beyond the current evidence. Describing this boundary precisely makes the HIL result useful rather than weak: it isolates and validates the detector’s numerical core.

C. Deployment Choice

Student A RF-KD is the preferred minimum-footprint configuration. It is the fastest board model and the most reliable Student A route. Student B RF-KD is the conservative higher-capacity deployment choice because it nearly matches Student B co-distillation while using a simpler training target and producing the existing fixed-point artifact. Student B co-distillation is the best mean ten-seed Student B result, but its higher variance and nonsignificant gain over scratch/RF-KD do not justify calling it uniformly superior.

VII. THREATS TO VALIDITY

A. Internal Validity

The primary historical threat is the archived WSN-DS scaler fit before splitting. Scaling is label-agnostic, yet its means and variances include the test distribution. Multi-seed WSN predictive tables, the SHAP audit pair, and Table X inherit that archived preprocessing lineage and must not be read as train-only estimates. A train-only-scaler seed-42 RF-KD confirmation regenerates deployment weights, ONNX/OpenVINO host runtime, fixed-point exports, and four full-test HIL pairs under train-fitted scaling; it is single-seed and does not replace the ten-seed distribution. A five-seed exact-feature-group package provides a descriptive leakage-control sensitivity check under group-disjoint partitions; it is not a statistical replacement for the archived multi-seed tables.

The ten seeds share one train/validation/test split, so variability estimates stochastic training rather than split uncertainty. KD hyperparameters were selected on the seed-42 validation route and then reused. The co-distillation and extended curriculum routes also use larger epoch budgets. The paired Wilcoxon family is Holm-adjusted, but inference remains low-powered at ten paired seeds and does not capture split uncertainty.

B. Construct Validity

Macro-F1 is primary but omits attack costs. Serialized RF size, FP32 parameter size, fixed-point constants, and linked firmware measure different constructs. Ratios across these bases are explicitly labeled. Reported compute ceilings exclude serial/radio transfer. Runtime benchmarks come from one host and support within-host comparisons.

SHAP agreement compares TreeExplainer and DeepExplainer outputs, so low agreement can reflect both model behavior and explainer/model-class differences. Only one final-seed Student A curriculum-KD pair was audited. Five repeated subsamples provide descriptive sensitivity summaries.

C. External Validity

WSN-DS is simulated and heavily dominated by Normal traffic. Edge-IIoTset broadens the attack taxonomy but also shows that results depend strongly on source file and retained fields. Live WSN efficacy remains outside both dataset studies. Development-board HIL validates the integer model core, while MSP430 evidence is cross-compile-only. Energy, radio integration, online feature extraction, temporal drift, and adversarial robustness remain unevaluated.

VIII. REPRODUCIBILITY AND ARTIFACT TRACEABILITY

All numerical table fragments are regenerated from versioned CSV, JSON, and compiler-log evidence. The generator verifies expected result counts, complete HIL sequences, and compiler totals, then records SHA-256 digests for every consumed source. Complete WSN route matrices, host-runtime rows, board replays, Edge-IIoT rankings, and build logs accompany the manuscript, together with a claim-to-artifact ledger. The implementation uses scikit-learn [44] and PyTorch [39]; venue-specific code and data availability text will identify the archival release used for review.

IX. CONCLUSION

CuKD-XAI shows that WSN intrusion-detector compression must be evaluated across prediction, explanation, numerical conversion, and execution. Under the archived WSN-DS protocol, 1,189- and 3,397-parameter students retain 0.9200 and 0.9335 mean macro-F1 in 4.64 and 13.27 KB of FP32 parameters. RF-KD is the strongest ultra-small route by mean macro-F1; co-distillation has the highest Student B mean, but no Student B paired comparison survives Holm adjustment. The audited curriculum-KD student's SHAP ranking does not align with the RF reference, and fixed-point conversion causes class-concentrated drift even though MCU execution exactly matches the integer reference. Four full-test replays and MSP430 cross-compilation establish executable model-core and memory-feasibility evidence, while Edge-IIoTset results expose dependence on protocol and capacity. The study therefore provides a measured compression-and-validation chain. Train-only seed-42 RF-KD confirmation closes the scaler-lineage gap for deployment and four-board HIL, and a five-seed feature-group package documents group-disjoint sensitivity; live feature extraction and energy evaluation remain necessary before complete-system deployment claims.

APPENDIX A

COMPLETE RESULT MATRICES

Tables XVII–XX contain every stored WSN-DS student route and every unique Edge-IIoTset configuration. The WSN-DS C and F aliases duplicate their historical nominal-schedule variants and are retained for traceability; shared Edge-IIoTset teacher rows duplicated under each student-capacity label in the source CSV are shown once.

TABLE XVII
ALL STUDENT A WSN-DS TEN-SEED RESULTS

Configuration	Acc. mean	Acc. std	Macro-F1 mean	Macro-F1 std	Params	Size KB
RF-500 teacher	0.9966	0.0001	0.9789	0.0003	–	85064.54
Full MLP	0.9873	0.0004	0.9232	0.0024	69,893	273.02
Domain curriculum MLP	0.9857	0.0005	0.9138	0.0032	69,893	273.02
Loss curriculum MLP (alias)	0.9859	0.0006	0.9147	0.0039	69,893	273.02
Loss curriculum MLP (extended)	0.9787	0.0238	0.8673	0.1548	69,893	273.02
Loss curriculum MLP (nominal schedule)	0.9859	0.0006	0.9147	0.0039	69,893	273.02
Scratch	0.9847	0.0021	0.9123	0.0062	1,189	4.64
MLP-KD	0.9861	0.0004	0.9164	0.0028	1,189	4.64
RF-KD	0.9869	0.0005	0.9200	0.0031	1,189	4.64
Curriculum-KD (alias)	0.9855	0.0003	0.9129	0.0021	1,189	4.64
Curriculum-KD (extended)	0.9836	0.0067	0.9064	0.0251	1,189	4.64
Curriculum-KD (nominal schedule)	0.9855	0.0003	0.9129	0.0021	1,189	4.64
Random pacing KD	0.9857	0.0005	0.9137	0.0028	1,189	4.64
SMOTE-teacher KD	0.9856	0.0003	0.9136	0.0019	1,189	4.64
RF+curriculum co-distillation	0.9865	0.0007	0.9181	0.0041	1,189	4.64

TABLE XVIII
ALL STUDENT B WSN-DS TEN-SEED RESULTS

Configuration	Acc. mean	Acc. std	Macro-F1 mean	Macro-F1 std	Params	Size KB
RF-500 teacher	0.9966	0.0001	0.9789	0.0003	–	85064.54
Full MLP	0.9873	0.0004	0.9232	0.0024	69,893	273.02
Domain curriculum MLP	0.9857	0.0005	0.9138	0.0032	69,893	273.02
Loss curriculum MLP (alias)	0.9859	0.0006	0.9147	0.0039	69,893	273.02
Loss curriculum MLP (extended)	0.9787	0.0238	0.8673	0.1548	69,893	273.02
Loss curriculum MLP (nominal schedule)	0.9859	0.0006	0.9147	0.0039	69,893	273.02
Scratch	0.9888	0.0006	0.9322	0.0038	3,397	13.27
MLP-KD	0.9881	0.0011	0.9275	0.0068	3,397	13.27
RF-KD	0.9891	0.0012	0.9328	0.0076	3,397	13.27
Curriculum-KD (alias)	0.9877	0.0006	0.9258	0.0034	3,397	13.27
Curriculum-KD (extended)	0.9855	0.0067	0.9172	0.0259	3,397	13.27
Curriculum-KD (nominal schedule)	0.9877	0.0006	0.9258	0.0034	3,397	13.27
Random pacing KD	0.9875	0.0009	0.9246	0.0055	3,397	13.27
SMOTE-teacher KD	0.9883	0.0014	0.9300	0.0083	3,397	13.27
RF+curriculum co-distillation	0.9891	0.0018	0.9335	0.0114	3,397	13.27

TABLE XIX
STRICT EDGE-IIOTSET FIVE-SEED RESULTS (43 FEATURES)

Capacity	Configuration	Acc. mean	Acc. std	Macro-F1 mean	Macro-F1 std	Params	Size KB
A (32–16)	RF-500 teacher	0.8833	0.0002	0.8729	0.0002	–	109879.57
B (64–32)	RF-KD	0.7309	0.0031	0.7111	0.0025	5,391	21.06
B (64–32)	RF+curriculum co-distillation	0.7304	0.0027	0.7100	0.0030	5,391	21.06
A (32–16)	RF+curriculum co-distillation	0.7243	0.0057	0.7010	0.0065	2,191	8.56
A (32–16)	Full MLP	0.7243	0.0040	0.6880	0.0037	74,511	291.06
B (64–32)	Scratch	0.7236	0.0011	0.6878	0.0018	5,391	21.06
A (32–16)	RF-KD	0.7109	0.0110	0.6863	0.0108	2,191	8.56
B (64–32)	Curriculum-KD (alias)	0.7198	0.0035	0.6860	0.0033	5,391	21.06
B (64–32)	MLP-KD	0.7189	0.0031	0.6850	0.0028	5,391	21.06
A (32–16)	Loss curriculum MLP (nominal schedule)	0.7214	0.0068	0.6825	0.0121	74,511	291.06
A (32–16)	MLP-KD	0.7121	0.0045	0.6772	0.0044	2,191	8.56
A (32–16)	Curriculum-KD (alias)	0.7077	0.0150	0.6713	0.0167	2,191	8.56
A (32–16)	Scratch	0.7001	0.0191	0.6646	0.0200	2,191	8.56

TABLE XX
LITERATURE-ORIENTED EDGE-IIOTSET FIVE-SEED RESULTS (49 FEATURES)

Capacity	Configuration	Acc. mean	Acc. std	Macro-F1 mean	Macro-F1 std	Params	Size KB
A (32–16)	RF-500 teacher	0.9825	0.0003	0.8892	0.0011	–	101114.94
A (32–16)	LightGBM teacher	0.9867	0.0001	0.8861	0.0003	–	15224.05
C (128–64)	RF-KD	0.9685	0.0002	0.8244	0.0009	15,631	61.06
C (128–64)	LightGBM-KD	0.9685	0.0007	0.8242	0.0031	15,631	61.06
C (128–64)	Scratch	0.9687	0.0001	0.8239	0.0003	15,631	61.06
B (64–32)	LightGBM-KD	0.9680	0.0002	0.8220	0.0010	5,775	22.56
C (128–64)	RF+curriculum co-distillation	0.9678	0.0010	0.8210	0.0028	15,631	61.06
B (64–32)	RF-KD	0.9672	0.0008	0.8197	0.0026	5,775	22.56
B (64–32)	Scratch	0.9679	0.0004	0.8189	0.0049	5,775	22.56
B (64–32)	RF+curriculum co-distillation	0.9666	0.0012	0.8169	0.0035	5,775	22.56
A (32–16)	LightGBM-KD	0.9661	0.0013	0.8143	0.0039	2,383	9.31
A (32–16)	RF+curriculum co-distillation	0.9658	0.0006	0.8127	0.0026	2,383	9.31
A (32–16)	RF-KD	0.9656	0.0010	0.8118	0.0032	2,383	9.31
A (32–16)	Scratch	0.9655	0.0011	0.8091	0.0039	2,383	9.31

REFERENCES

- [1] Y. Y. Ghadi, T. Mazhar, T. A. Shloul, T. Shahzad, U. A. Salaria, A. Ahmed, and H. Hamam, "Machine learning solutions for the security of wireless sensor networks: A review," *IEEE Access*, vol. 12, pp. 12 699–12 726, 2024.
- [2] M. A. Talukder, S. Sharmin, M. A. Uddin, M. M. Islam, and S. Aryal, "MLSTL-WSN: Machine learning-based intrusion detection using SMOTETomek in WSNs," *International Journal of Information Security*, vol. 23, no. 3, pp. 2139–2158, 2024.
- [3] M. A. Talukder, M. Khalid, and N. Sultana, "A hybrid machine learning model for intrusion detection in wireless sensor networks leveraging data balancing and dimensionality reduction," *Scientific Reports*, vol. 15, p. 4617, 2025.
- [4] S. A. Birahim, A. Paul, F. Rahman, Y. Islam, T. Roy, M. A. Hasan, F. Haque, and M. E. H. Chowdhury, "Intrusion detection for wireless sensor network using particle swarm optimization based explainable ensemble machine learning approach," *IEEE Access*, vol. 13, pp. 13 711–13 730, 2025.
- [5] V. K. Pandey, S. Prakash, T. K. Gupta, P. Sinha, T. Yang, R. S. Rathore, L. Wang, S. Tahir, and S. T. Bakhsh, "Enhancing intrusion detection in wireless sensor networks using a tabu search based optimized random forest," *Scientific Reports*, vol. 15, 2025.
- [6] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
- [7] J. Adebayo, J. Gilmer, M. Muehly, I. Goodfellow, M. Hardt, and B. Kim, "Sanity checks for saliency maps," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [8] S. Krishna, T. Han, A. Gu, J. Pombra, S. Jabbari, S. Wu, and H. Lakkaraju, "The disagreement problem in explainable machine learning: A practitioner's perspective," *Transactions on Machine Learning Research*, 2022.
- [9] I. Almomani, B. Al-Kasasbeh, and M. Al-Akhras, "WSN-DS: A dataset for intrusion detection systems in wireless sensor networks," *Journal of Sensors*, vol. 2016, p. 4731953, 2016.
- [10] T. M. Nguyen, H. H.-P. Vo, and M. Yoo, "Enhancing intrusion detection in wireless sensor networks using a GSWO-CatBoost approach," *Sensors*, vol. 24, no. 11, p. 3339, 2024.
- [11] M. Alqahtani, A. Gumaei, H. Mathkour, and M. M. B. Ismail, "A genetic-based extreme gradient boosting model for detecting intrusions in wireless sensor networks," *Sensors*, vol. 19, no. 20, p. 4383, 2019.
- [12] S. Salmi and L. Oughdir, "CNN-LSTM based approach for DoS attacks detection in wireless sensor networks," *International Journal of Advanced Computer Science and Applications*, vol. 13, no. 4, pp. 835–842, 2022.
- [13] X. Xiao and W. Duan, "Metaheuristically optimized deep soft-voting ensemble for explainable and resource-aware signal processing in wireless sensor network intrusion detection," *Signal, Image and Video Processing*, vol. 19, no. 15, 2025.
- [14] A. M. Alfarrar and A. A. AbuSamra, "Energy-efficient hybrid learning for secure wireless sensor networks," *ECTI Transactions on Computer and Information Technology*, vol. 19, no. 4, pp. 693–706, 2025.
- [15] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. 26th International Conference on Machine Learning*, 2009, pp. 41–48.
- [16] S. Yang, X. Zheng, Z. Xu, and X. Wang, "A lightweight approach for network intrusion detection based on self-knowledge distillation," *arXiv preprint arXiv:2307.10191*, 2023.
- [17] H. Benaddi, M. Jouhari, N. Laamech, A. Motii, and K. Ibrahim, "Lightweight intrusion detection in IoT via SHAP-guided feature pruning and knowledge-distilled kronecker networks," in *Proc. International Conference on Advanced Communication Technologies and Networking (CommNet)*, 2025, arXiv:2512.19488.
- [18] T. Wisanwanichthan and M. Thammawichai, "A lightweight intrusion detection system for IoT and UAV using deep neural networks with knowledge distillation," *Computers*, vol. 14, no. 7, p. 291, 2025.
- [19] M. A. Hossain, S. Saif, and M. S. Islam, "A novel federated learning approach for IoT botnet intrusion detection using SHAP-based knowledge distillation," *Complex & Intelligent Systems*, vol. 11, p. 422, 2025.
- [20] H. Peng, C. Wu, and Y. Xiao, "FD-IDS: Federated learning with knowledge distillation for intrusion detection in non-IID IoT environments," *Sensors*, vol. 25, no. 14, p. 4309, 2025.
- [21] M. A. Yagiz and P. Goktas, "LENS-XAI: Redefining lightweight and explainable network security through knowledge distillation and variational autoencoders for scalable intrusion detection in cybersecurity," *arXiv preprint arXiv:2501.00790*, 2025.
- [22] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [23] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [24] S. F. Misrak and H. M. Melaku, "Lightweight intrusion detection system for IoT with improved feature engineering and advanced dynamic quantization," *Discover Internet of Things*, vol. 5, 2025.
- [25] A. Javed, M. N. Awais, A. ul Haq Qureshi, M. Jawad, J. Arshad, and H. Larijani, "Embedding tree-based intrusion detection system in smart thermostats for enhanced IoT security," *Sensors*, vol. 24, no. 22, p. 7320, 2024.
- [26] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

- [27] B. Nugraha, A. V. Jnanashree, and T. Bauschert, "A versatile XAI-based framework for efficient and explainable intrusion detection systems," *Annals of Telecommunications*, vol. 80, pp. 1095–1120, 2025.
- [28] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022.
- [29] A. Diab, A. Chehade, E. Ragusa, P. Gastaldo, R. Zunino, A. Baghdadi, and M. Rizk, "Intrusion detection on resource-constrained IoT devices with hardware-aware ML and DL," *arXiv preprint arXiv:2512.02272*, 2025.
- [30] W. Gao, M. Wang, Y. Pei, F. Li, and C. Wang, "A lightweight multi-classification intrusion detection model for edge IoT networks," *Electronics*, vol. 15, no. 5, p. 938, 2026.
- [31] A. A. Seyedkolaei, F. Mahmoudi, and J. Garcia, "A deep learning approach for multiclass attack classification in IoT and IIoT networks using convolutional neural networks," *Future Internet*, vol. 17, no. 6, p. 230, 2025.
- [32] T. Hasan, A. Hossain, M. Q. Ansari, and T. H. Syed, "Enhanced intrusion detection in IIoT networks: A lightweight approach with autoencoder-based feature learning," in *Proc. 10th International Conference on Internet of Things, Big Data and Security*, 2025, pp. 207–214.
- [33] M. S. Alshehri, O. Saidani, F. S. Alrayes, S. F. Abbasi, and J. Ahmad, "A self-attention-based deep convolutional neural networks for IIoT networks intrusion detection," *IEEE Access*, vol. 12, pp. 45 762–45 772, 2024.
- [34] P. K. Badhan, "Neuro-symbolic machine learning for lightweight and interpretable IoT edge intrusion detection," *Discover Sensors*, vol. 2, p. 15, 2026.
- [35] F. Adjewa, M. Esseghir, and L. Merghem-Boulaiahia, "From edge transformer to IoT decisions: Offloaded embeddings for lightweight intrusion detection," *Sensors*, vol. 26, no. 2, p. 356, 2026.
- [36] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.
- [37] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th International Conference on Machine Learning*, vol. 70, 2017, pp. 1321–1330.
- [38] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [39] A. Paszke, S. Gross, F. Massa *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [40] ONNX Community, "Open neural network exchange," <https://onnx.ai/>, accessed: 2026-08-03.
- [41] Microsoft, "ONNX Runtime," <https://onnxruntime.ai/>, accessed: 2026-08-03.
- [42] Intel, "OpenVINO Toolkit," <https://docs.openvino.ai/>, accessed: 2026-08-03.
- [43] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [44] F. Pedregosa, G. Varoquaux, A. Gramfort *et al.*, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.